# Grokking and Mechanistic Interpretability

## How a tiny transformer learns modular addition with Fourier features

Power et al. (2022) Nanda et al. (2023) Welch Labs (2025)

# Roadmap

1. The puzzle: why test performance can improve long after memorization.
2. Generic transformer mechanics: how the $=$ position gathers and transforms information.
3. The identified circuit: Fourier features make modular addition easy to express.
4. Beyond the toy model: manifolds, superposition, and sparse autoencoders.

**Running example:** $P = 7$, so the prompt 2 3 $=$ should produce 5.

# The puzzle

**A model can memorize first and understand later.**

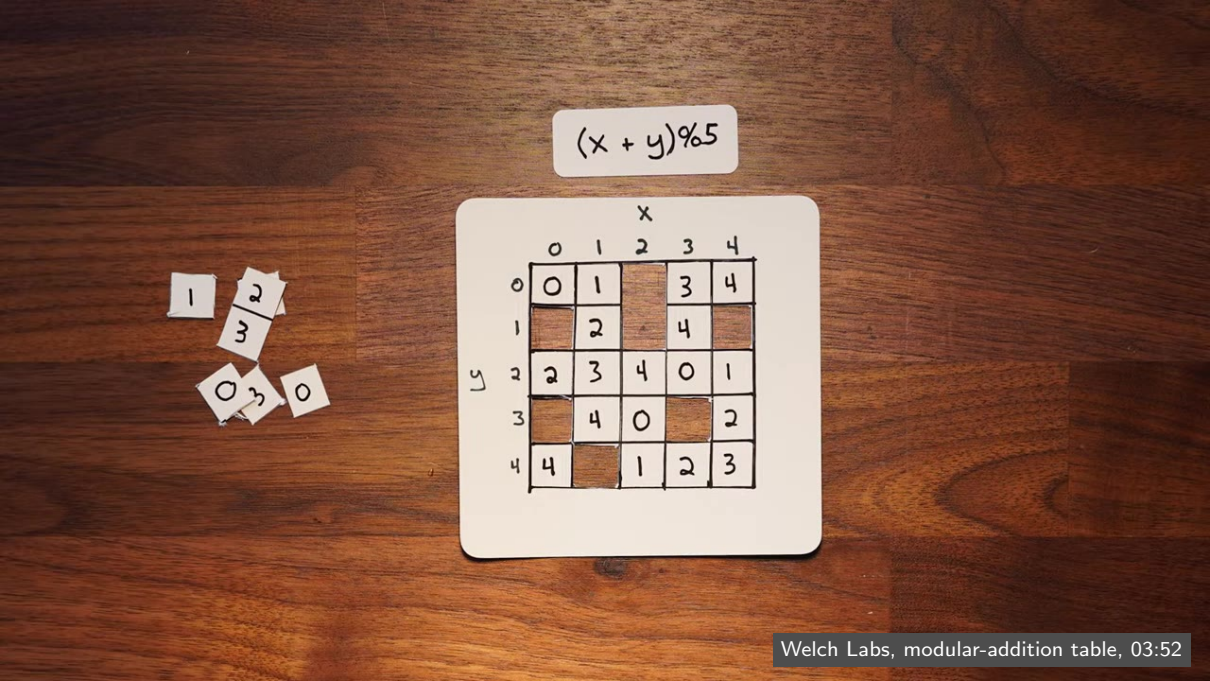
# Predict first: is training accuracy the whole story?


**Grokking:** delayed generalization after apparent overfitting.

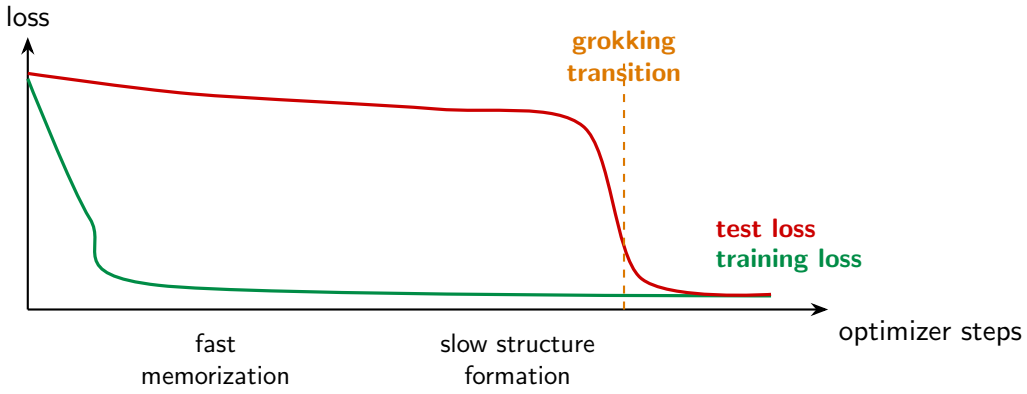
# The task: fill in an incomplete modular-addition table

- Work modulo $P$: after $P-1$, wrap back to 0.
- Running example with $P = 7$: $2 + 3 = 5 \pmod 7$.
- Train on a random subset of pairs $(a, b)$.
- Test on the missing cells of the table.

| $+ \bmod 7$ | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
| 1 | 1 | 2 | 3 | 4 | 5 | 6 | 0 |
| 2 | 2 | 3 | 4 | 5 | 6 | 0 | 1 |
| 3 | 3 | 4 | 5 | 6 | 0 | 1 | 2 |
| 4 | 4 | 5 | 6 | 0 | 1 | 2 | 3 |


(x + y)%5
X
0 1 2 3 4
y
0 0 1 3 4
1 2 4
2 2 3 4 0 1
3 4 0 2
4 4 1 2 3
1 2 3 0 3 0

# What the classic grokking curve means




This is a conceptual learning curve: the point is the gap between fast fitting and late held-out improvement.


Welch Labs, grokking learning curve, 07:56

# Generic transformer mechanics

**Follow $2 \quad 3 \quad =$ from symbols to a prediction. The next section identifies the particular circuit this model learns.**

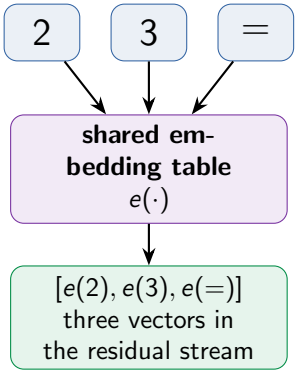
# The whole computation, at one glance


The special output position

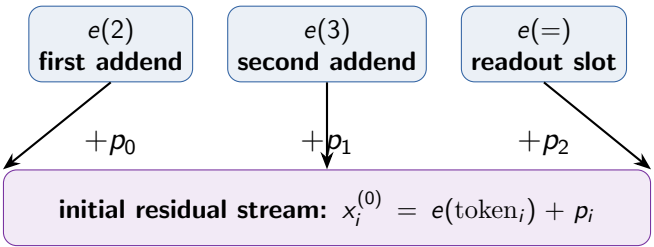
For our running input 2 3 =, the model reads the residual stream at = and should assign the largest logit to $c = 5$.

# Step 1: tokens become vectors

- The vocabulary contains the residues and symbols such as $=$.
- A learned table maps each token to a vector: $e(n), e(=) \in \mathbb{R}^{d_{\text{model}}}$.
- For the running prompt, lookup produces $e(2), e(3), e(=)$.
- At this stage, token IDs are just labels. No addition rule is built in.



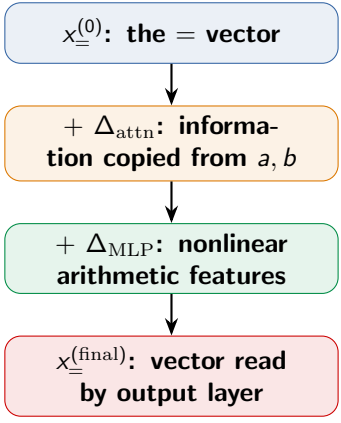
# Step 2: position says which role each token plays

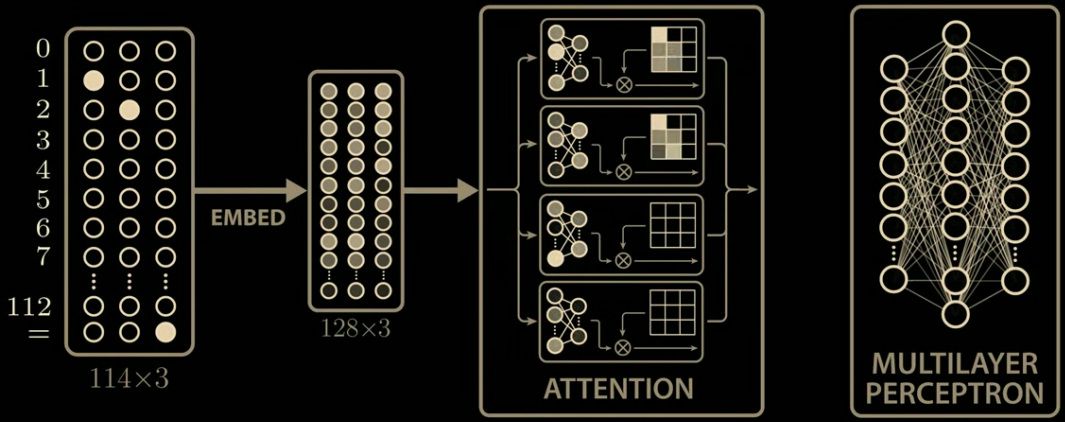



Position vectors matter because 2, 3, and $=$ must receive different computational roles even when all are tokens.

# The residual stream is the model's shared workspace

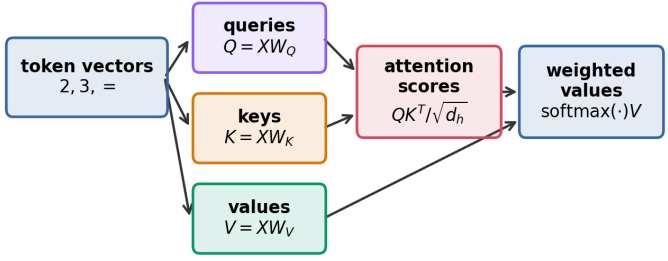
- Every layer reads the current vector at each position.
- Attention and the MLP each write an update back into the same space.
- The additions are literal: $x \leftarrow x + \Delta_{\text{attn}} + \Delta_{\text{MLP}}$.
- Mechanistic interpretability tracks what each update adds to the answer position.






Welch Labs, attention and residual stream, 10:20

# Step 3: attention routes 2 and 3 into the equals position




At the '=' position, one head chooses which token information should be written into the residual stream.

In this generic schematic, only the query at = matters: it should fetch both operand vectors into one workspace.

# Attention is a data-dependent weighted sum

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V$$

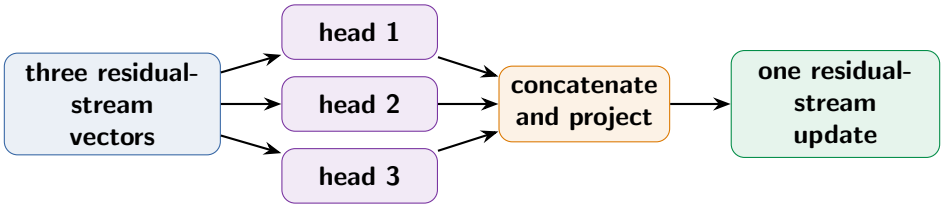
$$A = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_h}}\right) \qquad \Delta_{\mathrm{attn}} = AVW_O$$

- Row $A_{=,:}$ determines what the $=$ position reads.
- Large weights on 2 and 3 route both addends into one workspace.
- $W_O$ returns each head's result to the residual stream.

| | 2 | 3 | = |
|---|---|---|---|
| 2 | .60 | .35 | .05 |
| 3 | .20 | .55 | .25 |
| = | .45 | .45 | .10 |

Illustrative only. The highlighted bottom row is the read at $=$.

# Multiple heads are parallel communication channels

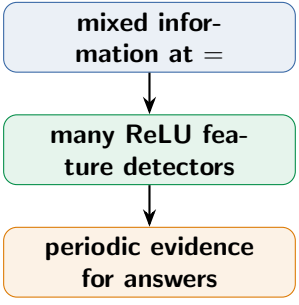



A small modular-addition model is much simpler than a modern language model, but the same Q/K/V communication pattern is present.

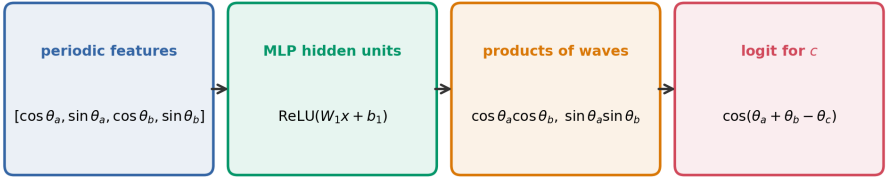
# Step 4: the MLP makes nonlinear features

$$h = \mathrm{ReLU}(W_{\mathrm{in}}x + b_{\mathrm{in}}), \quad \Delta_{\mathrm{MLP}} = W_{\mathrm{out}}h + b_{\mathrm{out}}$$

- Each hidden neuron is a learned detector on the current residual vector.
- ReLU gates that detector on or off.
- A weighted sum of many hidden units can approximate nonlinear functions of the two inputs.
- Here, the useful nonlinear evidence should support the candidate answer 5.



# The MLP converts periodic input features into answer evidence

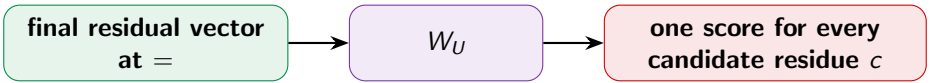



The MLP is not ``adding'' symbols directly. It builds nonlinear products that implement the trig addition identity.

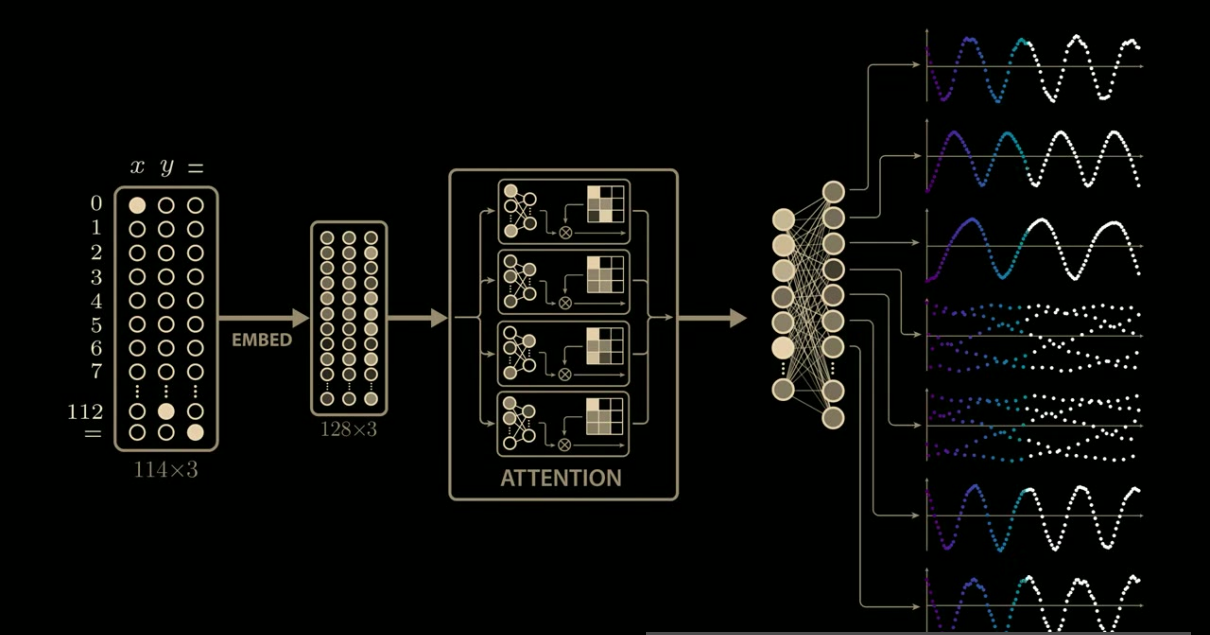
The diagram is a functional description. A ReLU MLP approximates these nonlinear combinations through many piecewise-linear hidden units; it is not a literal multiplication gate.

# Step 5: an unembedding turns the final vector into logits

$$\text{logits} = x_{=}^{(\text{final})} W_U + b_U \qquad \Longrightarrow \qquad \Pr(c \mid a, b) = \text{softmax}(\text{logits})_c$$




For $2 + 3$, interpretability can project an intermediate update through $W_U$ and ask: does it increase the logit for 5?

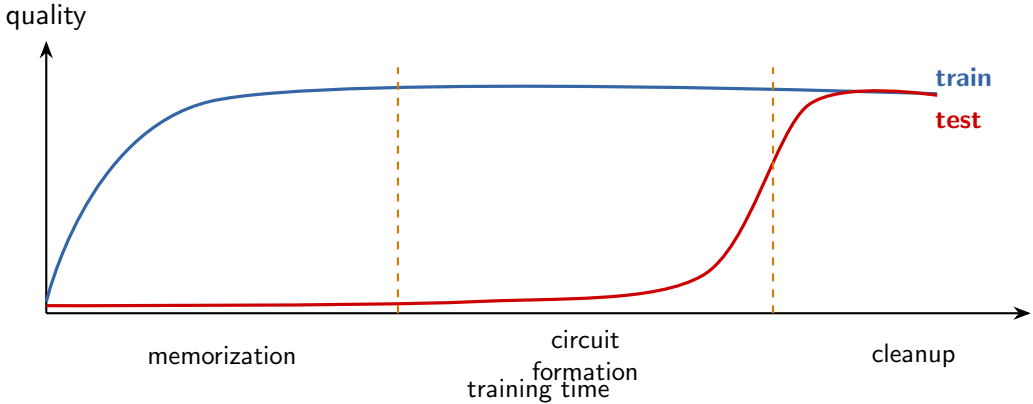



Welch Labs, transformer and periodic activations, 12:10

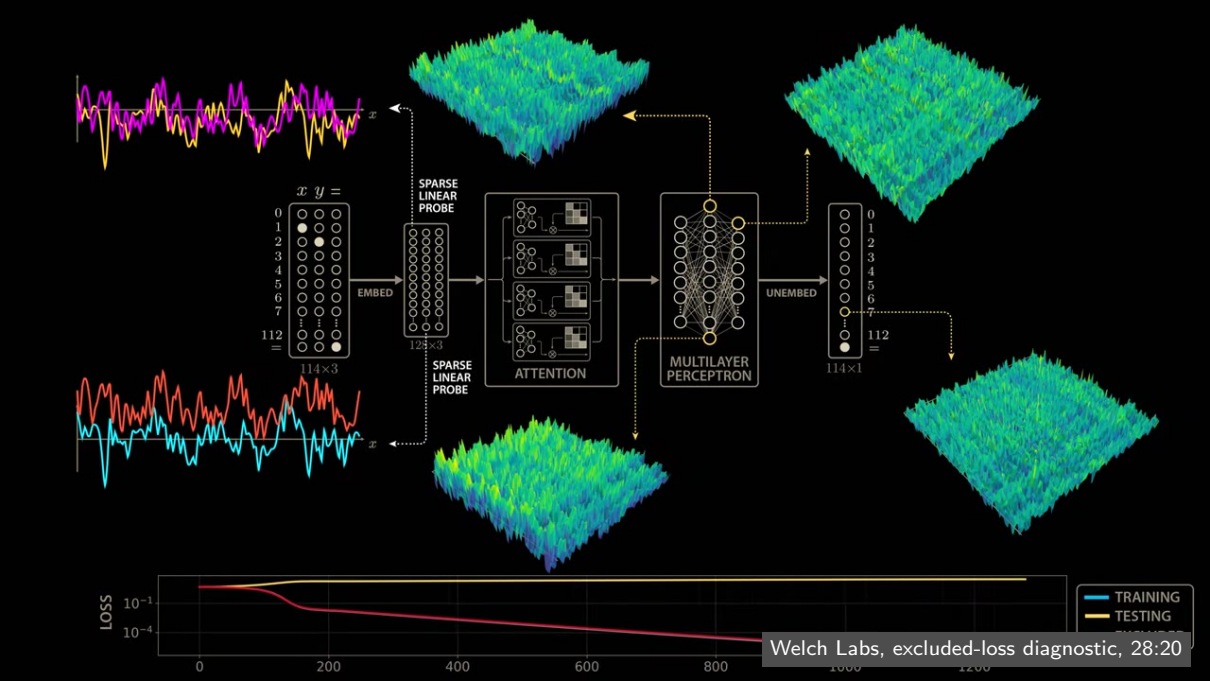
# The identified modular-addition circuit

**We now leave generic transformer mechanics and ask what this trained model actually uses to solve modular addition.**

# Three useful phases of grokking

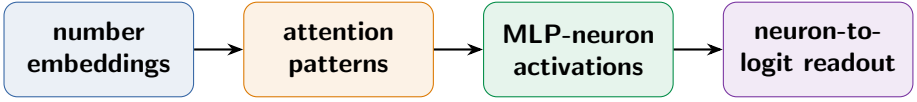



Nanda et al. (2023): hidden progress can precede the visible accuracy transition.

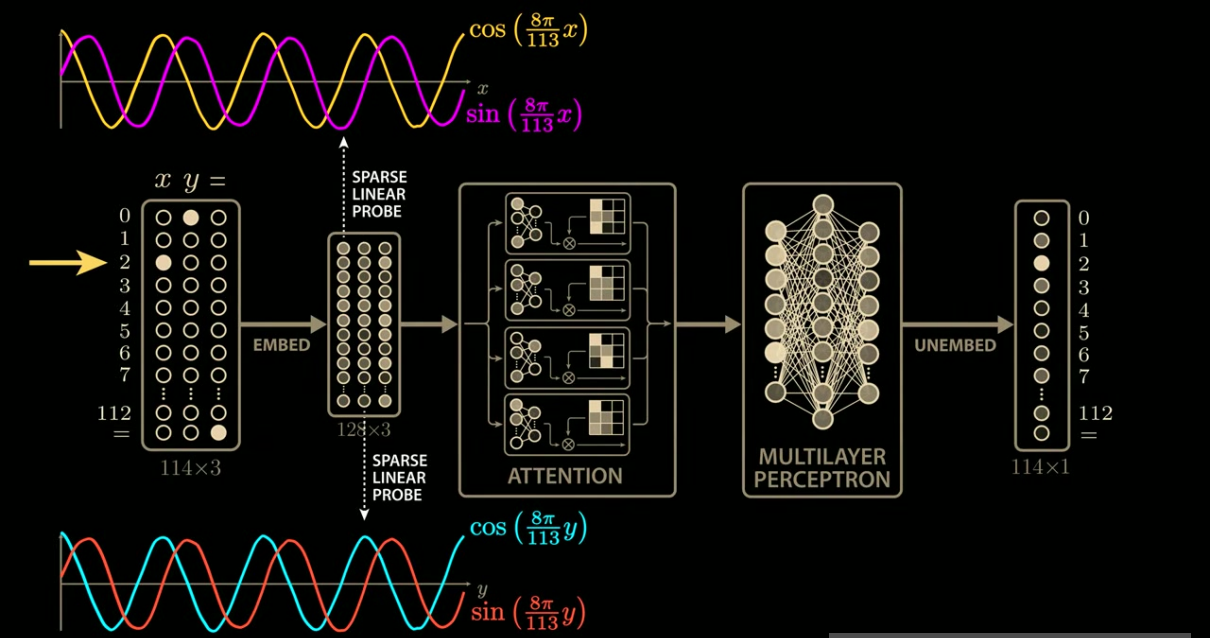



Welch Labs, excluded-loss diagnostic, 28:20

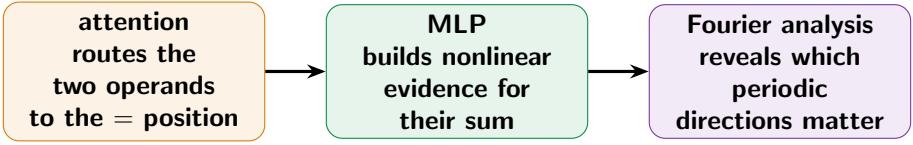
# What should we inspect inside the network?




Look for periodic structure, then test whether removing it breaks the answer.


$\cos\left(\frac{8\pi}{113}x\right)$
$x$
$\sin\left(\frac{8\pi}{113}x\right)$
$x$ $y$ =
0
1
2
3
4
5
6
7
112
=
114×3
EMBED
SPARSE LINEAR PROBE
128×3
SPARSE LINEAR PROBE
ATTENTION
MULTILAYER PERCEPTRON
UNEMBED
0
1
2
3
4
5
6
7
112
=
114×1
$\cos\left(\frac{8\pi}{113}y\right)$
$y$
$\sin\left(\frac{8\pi}{113}y\right)$

# Checkpoint: what each part must contribute

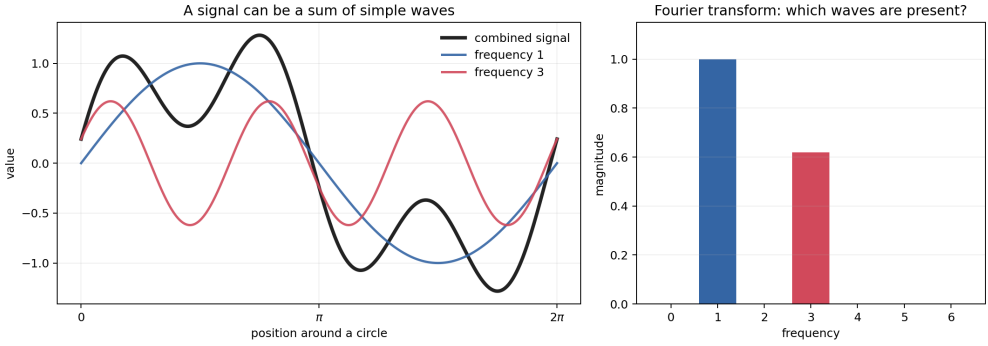



For $2 + 3$, the circuit should create evidence specifically for 5.

# Why Fourier transforms?

**They reveal circular patterns that are hard to see in raw coordinates.**

# A Fourier transform asks: which simple waves make up this signal?

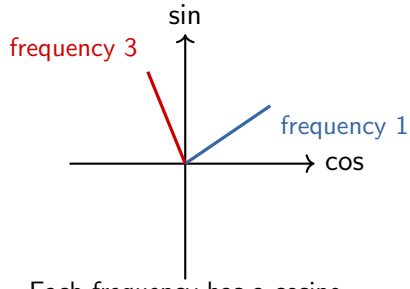



Instead of describing a signal point-by-point, describe how strongly each frequency is present.

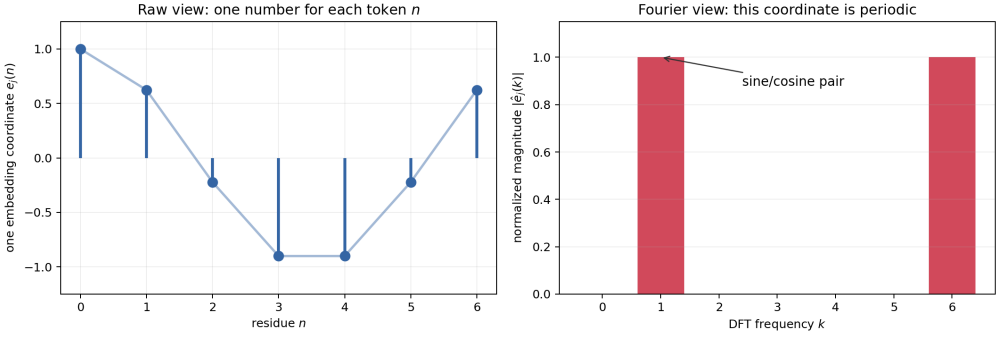
# A concrete example: a two-wave signal

$$f(x) = \sin x + 0.62 \sin(3x + 0.4)$$

- The raw curve looks irregular because two motions are superposed.
- The Fourier view separates it into frequency 1 and frequency 3.
- The height and phase of each component summarize the whole signal.



# Bridge: the DFT receives one value for every residue token




For an embedding coordinate $e_j$, treat $e_j(0), \ldots, e_j(P-1)$ as a discrete signal over token IDs. The DFT asks whether it traces a simple cycle around the modular circle.

# For modular arithmetic, use the discrete Fourier transform

$$\widehat{x}_k = \sum_{n=0}^{P-1} x_n \, e^{-2\pi i k n / P}, \qquad k = 0, \ldots, P-1$$

**Input**

Take a quantity defined for each residue $n \in \{0, \ldots, P-1\}$: an embedding coordinate, neuron activation, or output logit.

**Output**

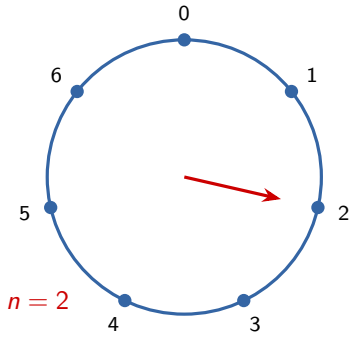
For each frequency $k$, $\widehat{x}_k$ records the strength and phase of a wave as we walk around the modular circle.

In a real-valued model, the useful basis appears as sine/cosine pairs rather than complex numbers.

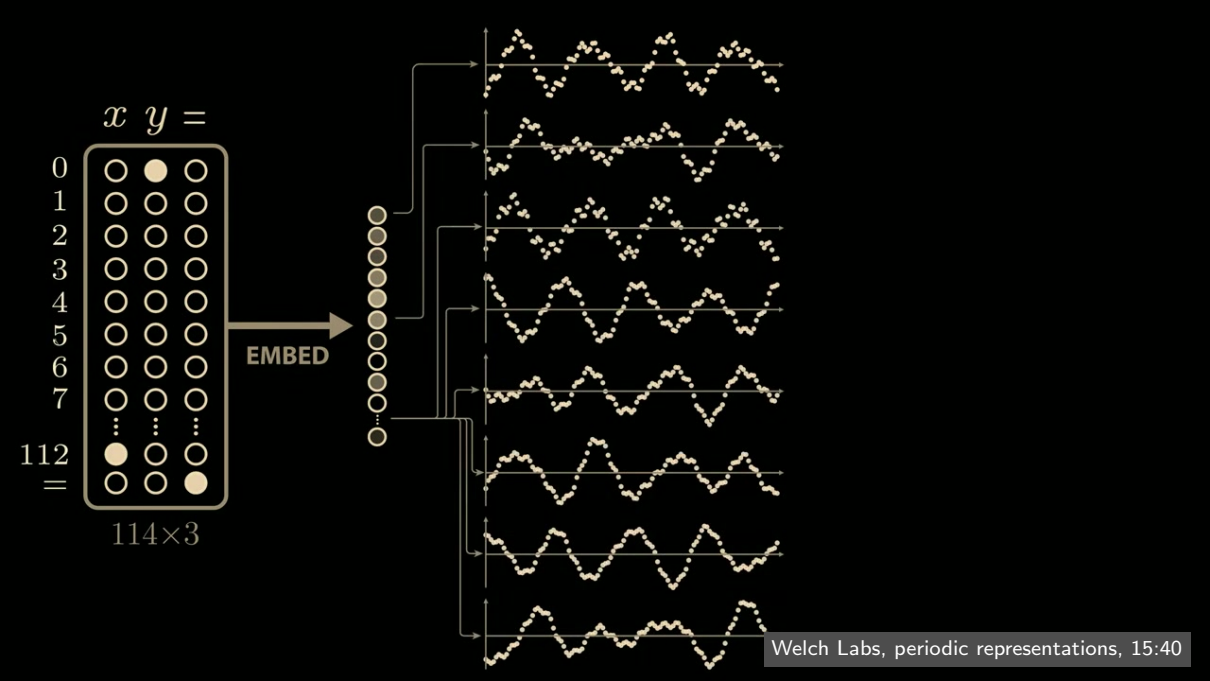
# Why a sine/cosine pair is a circular coordinate system

$$n \longmapsto \left(\cos\frac{2\pi kn}{P}, \sin\frac{2\pi kn}{P}\right)$$

- $k$ says how many turns the wave completes around the residue circle.
- The pair retains phase: where the wave is on the circle.
- Addition of residues becomes addition of angles.

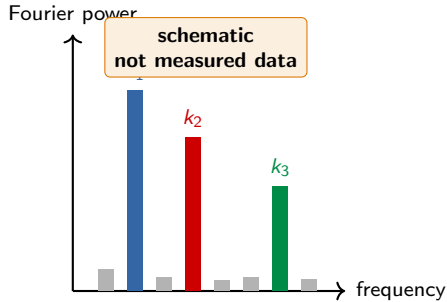



A residue is represented by a point on a rotating Fourier circle.


x y =
0
1
2
3
4
5
6
7
112
=
114×3
EMBED

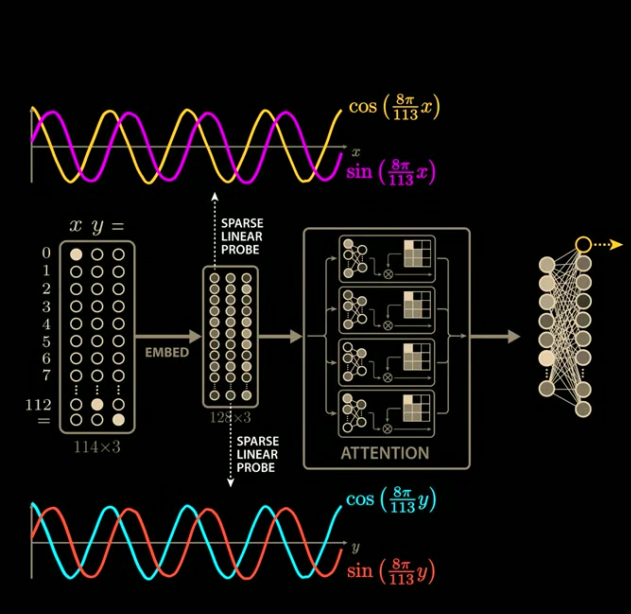
# The mechanistic finding: a small set of Fourier modes dominates

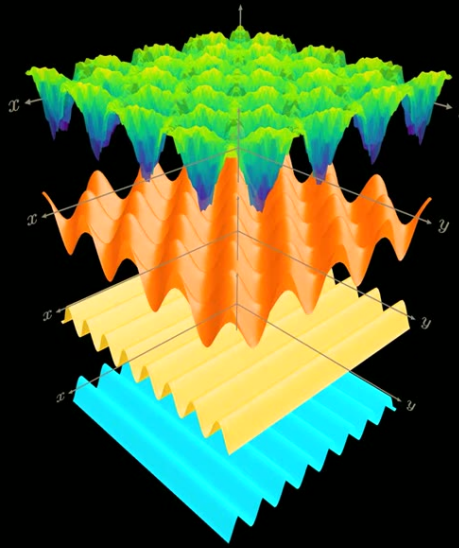
- Fourier analysis reveals periodic directions in embeddings, MLP activations, and logits.
- Only a few frequencies carry most of the modular-addition computation.
- This turns an enormous parameter set into a compact candidate circuit.



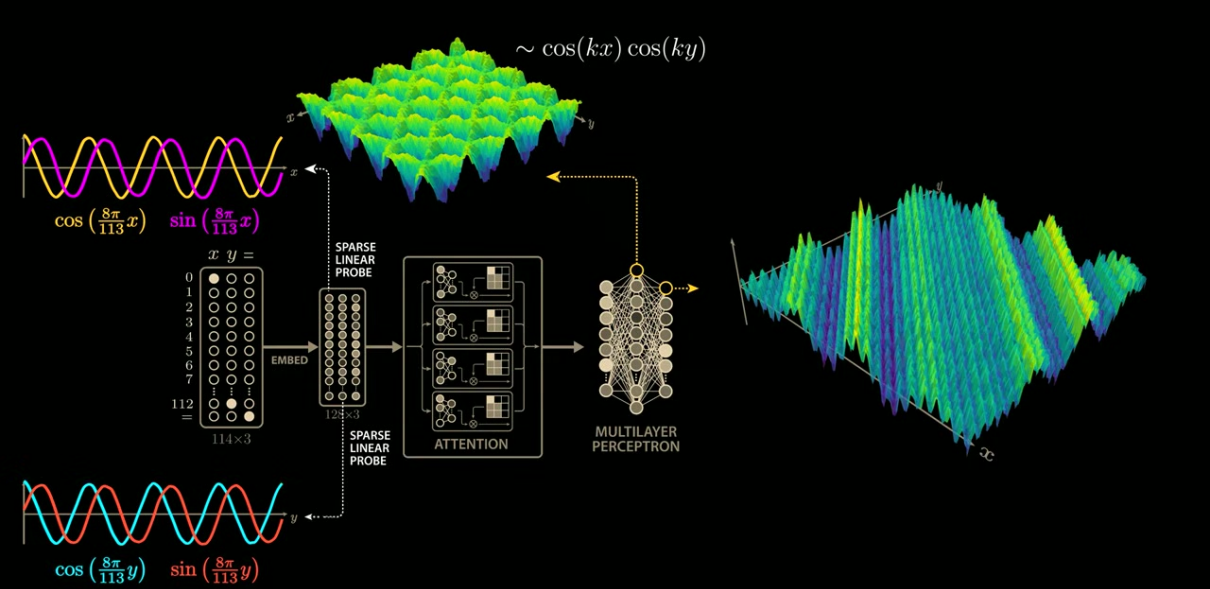

A few dominant frequencies form a candidate circuit.

Nanda et al. (2023) use Fourier structure together with causal interventions, not Fourier plots alone.



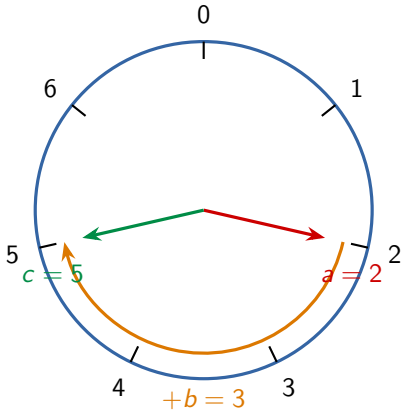

Welch Labs, probes and frequency surfaces, 20:30


$\sim \cos(kx)\cos(ky)$
$\cos\left(\frac{8\pi}{113}x\right)$
$\sin\left(\frac{8\pi}{113}x\right)$
$x$ $y$ =
0
1
2
3
4
5
6
7
112
=
114×3
EMBED
SPARSE LINEAR PROBE
128×3
SPARSE LINEAR PROBE
ATTENTION
MULTILAYER PERCEPTRON
$\cos\left(\frac{8\pi}{113}y\right)$
$\sin\left(\frac{8\pi}{113}y\right)$

# A clock turns addition into rotation

**The Fourier basis makes the arithmetic rule geometrically simple.**

# Numbers modulo $P$ live naturally on a clock




Rotation view

Represent $n$ by the angle $2\pi n/P$.

$$\theta_{2+3} = \theta_2 + \theta_3 \pmod{2\pi}$$

Adding residues becomes adding angles around a circle.


Welch Labs, clock analogy, 17:30

# The trigonometric identity that composes rotations

$$\cos(x + y) = \cos x \cos y - \sin x \sin y$$

$$\sin(x + y) = \sin x \cos y + \cos x \sin y$$

Input features

Encode $a$ and $b$ as $(\cos\theta_a, \sin\theta_a)$ and $(\cos\theta_b, \sin\theta_b)$.

Answer score

For candidate $c$, a natural score is $\cos(\theta_a + \theta_b - \theta_c)$, highest when $c = a + b$.

# Worked score: why $c = 5$ wins for $2 + 3$ modulo 7

$$\theta_n = \frac{2\pi n}{7}$$

$$\theta_2 = \frac{4\pi}{7}, \qquad \theta_3 = \frac{6\pi}{7}, \qquad \theta_5 = \frac{10\pi}{7}$$

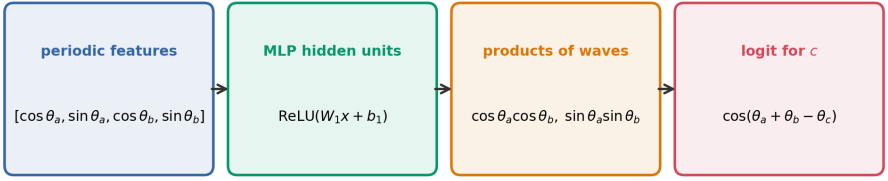
$$\text{score}(c) = \cos(\theta_2 + \theta_3 - \theta_c)$$

$$\text{score}(5) = \cos(0) = 1$$

$$\text{score}(4) = \cos\left(\frac{2\pi}{7}\right) < 1$$

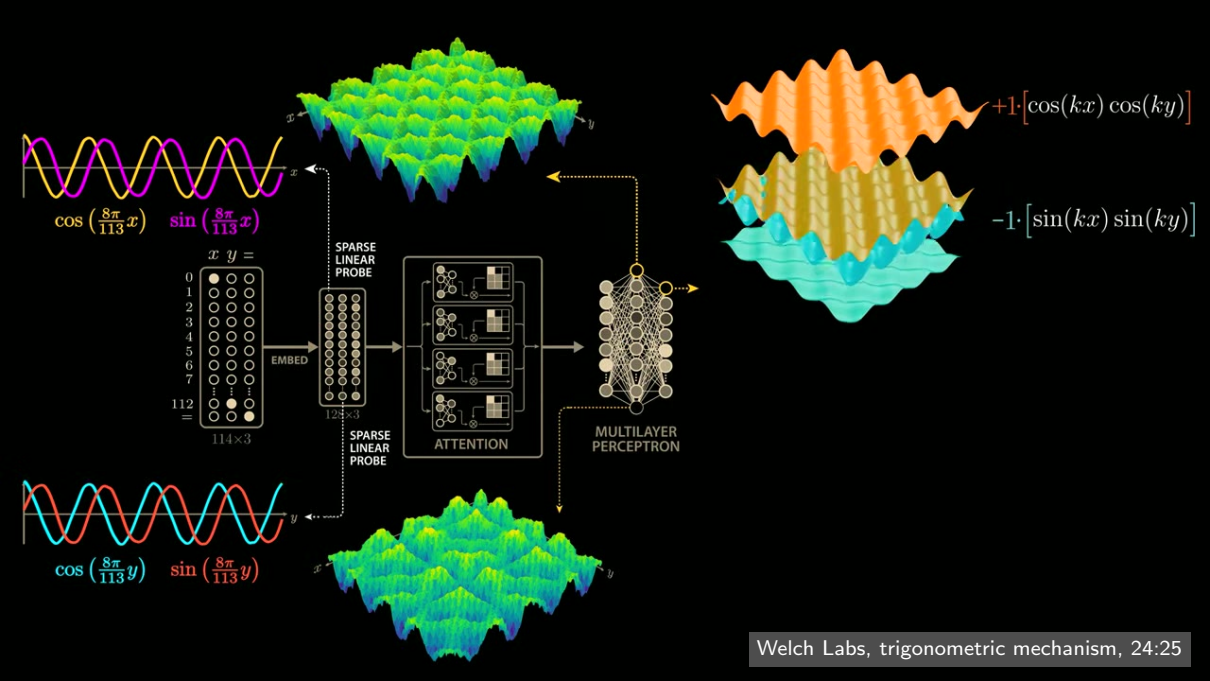
Attention brings periodic features for 2 and 3 together. The MLP approximates the products in the trig identity, and the unembedding compares the resulting phase with every candidate $c$.

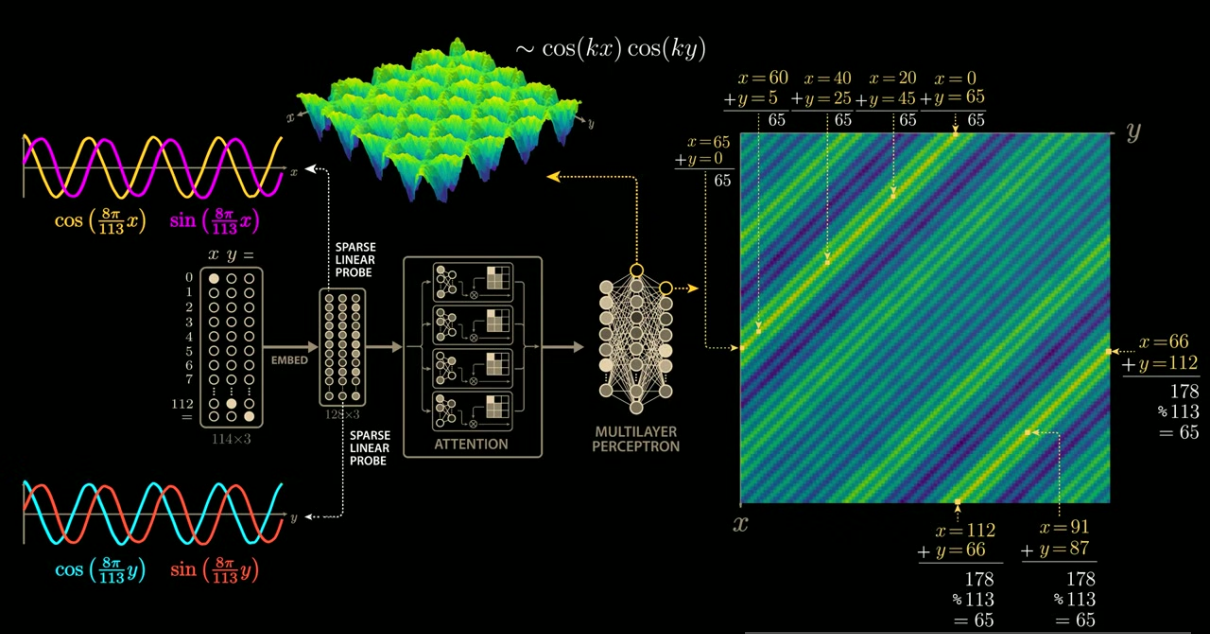
# A Fourier-style circuit for modular addition




The MLP is not ``adding'' symbols directly. It builds nonlinear products that implement the trig addition identity.

For every useful frequency, the network can vote for the candidate whose phase matches the summed input phase. Several frequencies make that vote more specific.

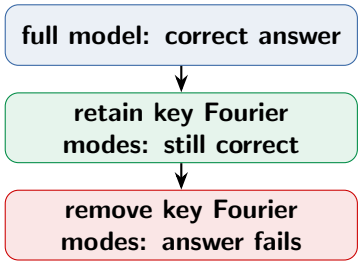

$\cos\left(\frac{8\pi}{113}x\right)$
$\sin\left(\frac{8\pi}{113}x\right)$
$x$ $y$ =
0
1
2
3
4
5
6
7
112
=
114×3
EMBED
SPARSE LINEAR PROBE
128×3
SPARSE LINEAR PROBE
ATTENTION
MULTILAYER PERCEPTRON
$+1\cdot\left[\cos(kx)\cos(ky)\right]$
$-1\cdot\left[\sin(kx)\sin(ky)\right]$
$\cos\left(\frac{8\pi}{113}y\right)$
$\sin\left(\frac{8\pi}{113}y\right)$



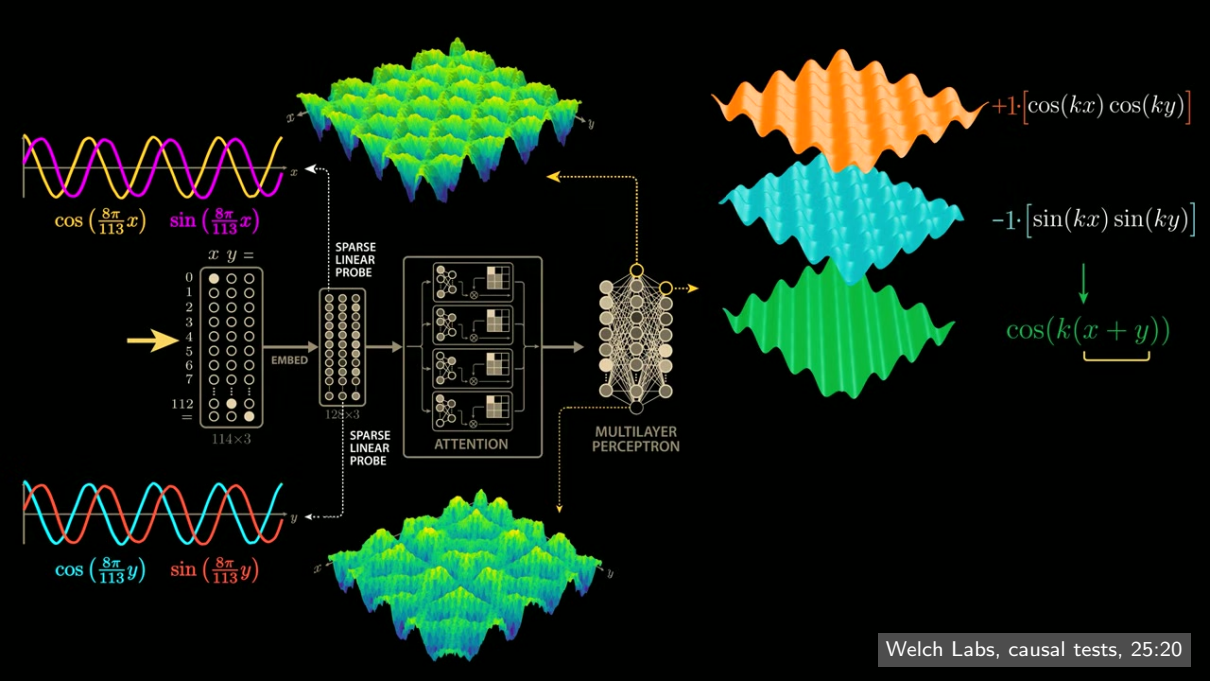

Welch Labs, circuit-level interpretation, 23:10

# A causal explanation needs interventions

- Fourier-transform embeddings and logits.
- Identify a few key frequencies.
- Keep only those frequencies: performance remains.
- Remove them: performance collapses.




Nanda et al. (2023): this is the evidence standard. A periodic-looking plot alone does not establish a mechanism.

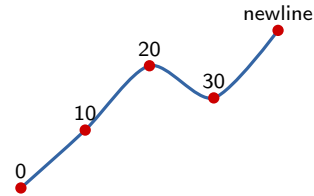

$\cos\left(\frac{8\pi}{113}x\right)$
$\sin\left(\frac{8\pi}{113}x\right)$
$x$
$y$
$x\ y =$
0
1
2
3
4
5
6
7
112
=
114×3
EMBED
SPARSE LINEAR PROBE
128×3
SPARSE LINEAR PROBE
ATTENTION
MULTILAYER PERCEPTRON
$\cos\left(\frac{8\pi}{113}y\right)$
$\sin\left(\frac{8\pi}{113}y\right)$
$+1\cdot[\cos(kx)\cos(ky)]$
$-1\cdot[\sin(kx)\sin(ky)]$
$\cos(k(x+y))$

# From a toy model to Claude

**Different task, different evidence: this is not a second grokking result. The interpretability questions transfer, but the mechanisms are less neatly localized.**

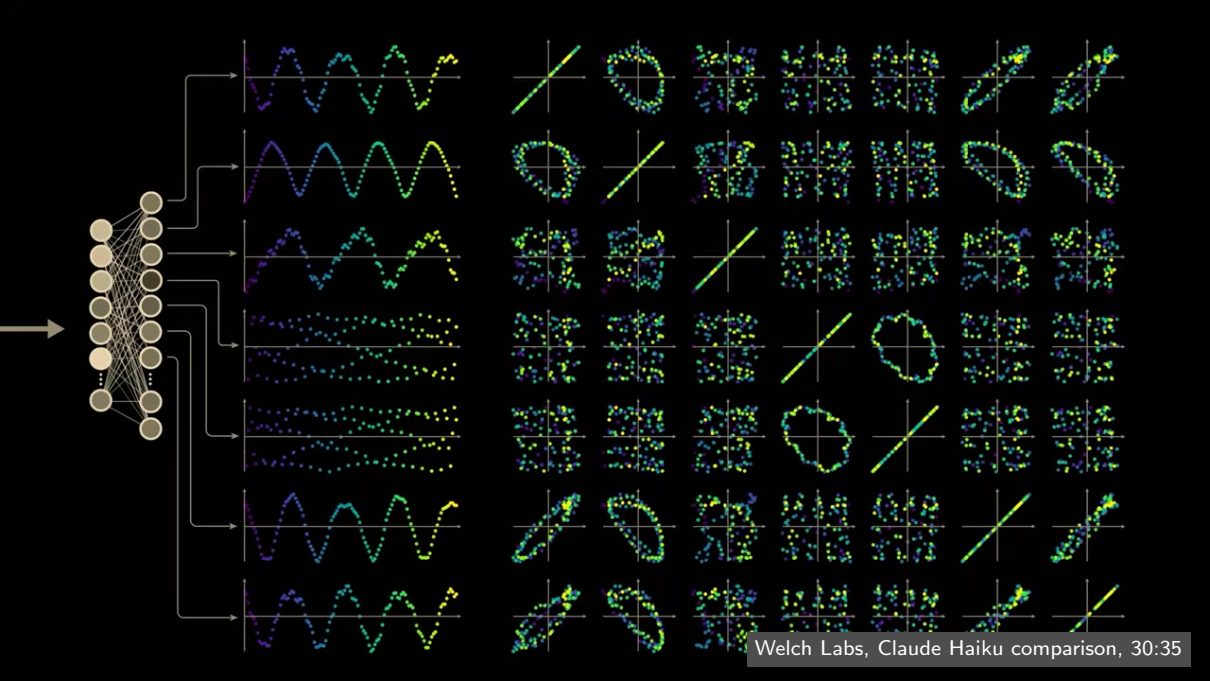
# Claude Haiku: a line-break counting manifold

- Claude 3.5 Haiku adapts to different target line widths.
- Internal representations track position within the line and the width target.
- The relevant states form a geometric counting manifold, not one isolated newline neuron.



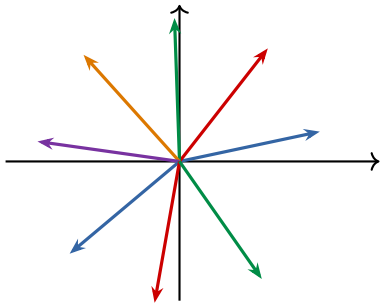

A continuous internal trajectory supports a discrete line-break decision.

Gurnee et al. (2025), *When Models Manipulate Manifolds*.


Welch Labs, Claude Haiku comparison, 30:35

Welch Labs, feature geometry, 31:30

# Superposition: many feature directions, few dimensions

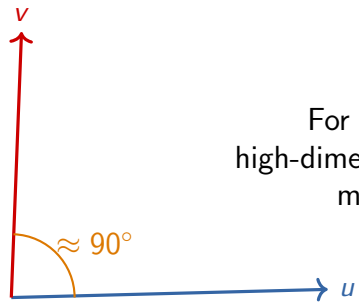


Many features can share a representation without being exactly orthogonal.

- A single activation dimension may help encode several features.
- In high dimensions, many directions can have small pairwise overlap.
- The price is polysemanticity: one neuron can respond to different concepts in different contexts.

Elhage et al. (2022), *Toy Models of Superposition*.

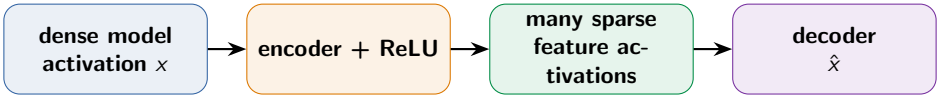
# Why near-right angles matter




For a fixed tolerance around 90°,
high-dimensional spaces admit exponentially
many low-overlap directions.

Nearly orthogonal does not mean independent. It means interference can be small enough for sparse features to coexist.

# Response: sparse autoencoders recover feature-like directions




$$\text{loss} = \|x - \hat{x}\|^2 + \lambda \|f(x)\|_1$$

If individual neurons are polysemantic, a sparse feature dictionary gives a more useful unit of interpretation. Bricken et al. (2023) reconstruct activations while encouraging only a few active features.

# Takeaways

**Interpretability asks for mechanisms, not just visual patterns.**

# Recap

1. Grokking separates fast memorization from delayed generalization.
2. A transformer moves addend information into the $=$ residual stream, transforms it through an MLP, then reads answer logits.
3. Fourier pairs make modular addition geometrically simple: adding residues is adding phases.
4. A real explanation requires causal tests, then richer feature tools for larger models.

**Question to carry forward:** Which internal directions are necessary for the model's answer?

# Sources and visual credits


- Power et al. (2022), *Grokking: Generalization Beyond Overfitting on Small Algorithmic Datasets*.
- Nanda et al. (2023), *Progress Measures for Grokking via Mechanistic Interpretability*.
- Gurnee et al. (2025), *When Models Manipulate Manifolds: The Geometry of a Counting Task*.
- Bricken et al. (2023), *Towards Monosemanticity: Decomposing Language Models With Dictionary Learning*.
- Elhage et al. (2022), *Toy Models of Superposition*.
- Welch Labs video stills: *The most complex model we actually understand*, 2025.